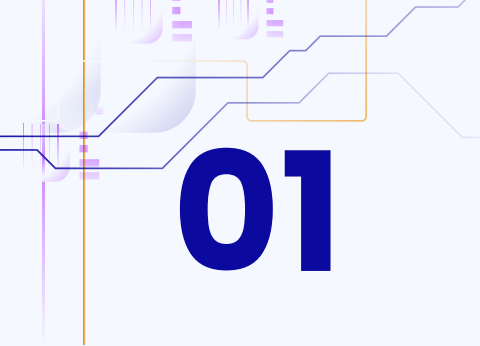




Architektura Aplikacji StudyHub

Wiktor Rzepka, Aleksandra Suder, Marcin Stępkowski



01

Widok logiczny (UML Class Diagram)



Ogólna struktura klas i zależności systemu StudyHub

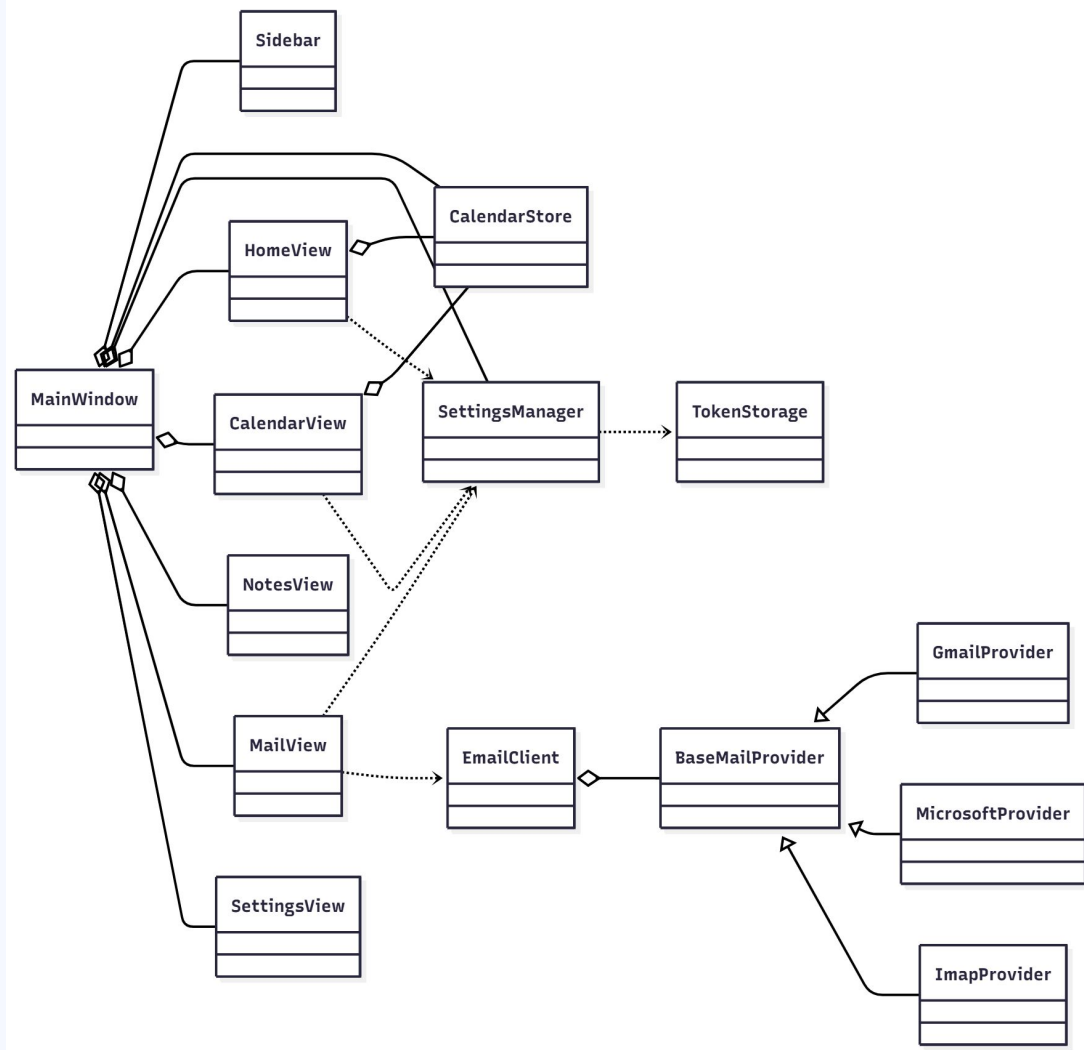
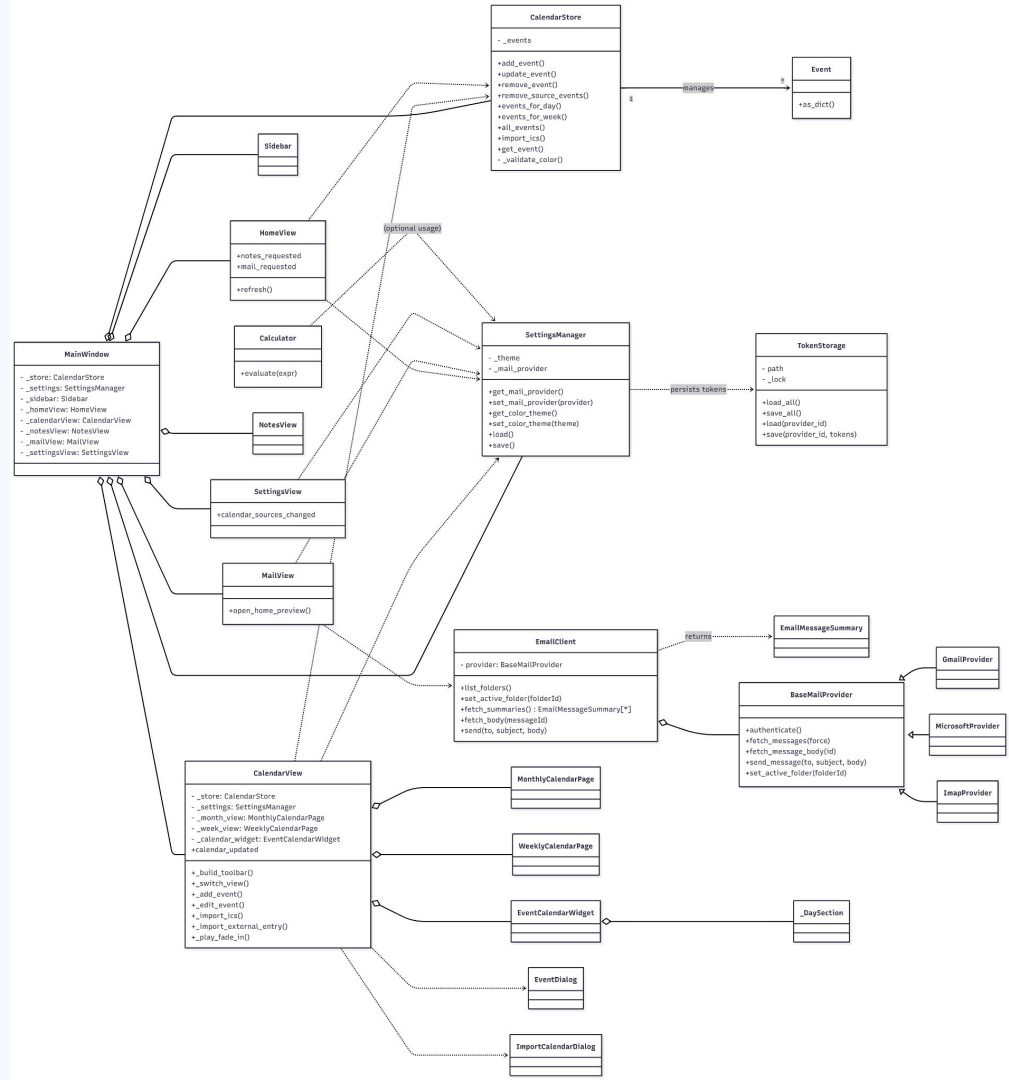


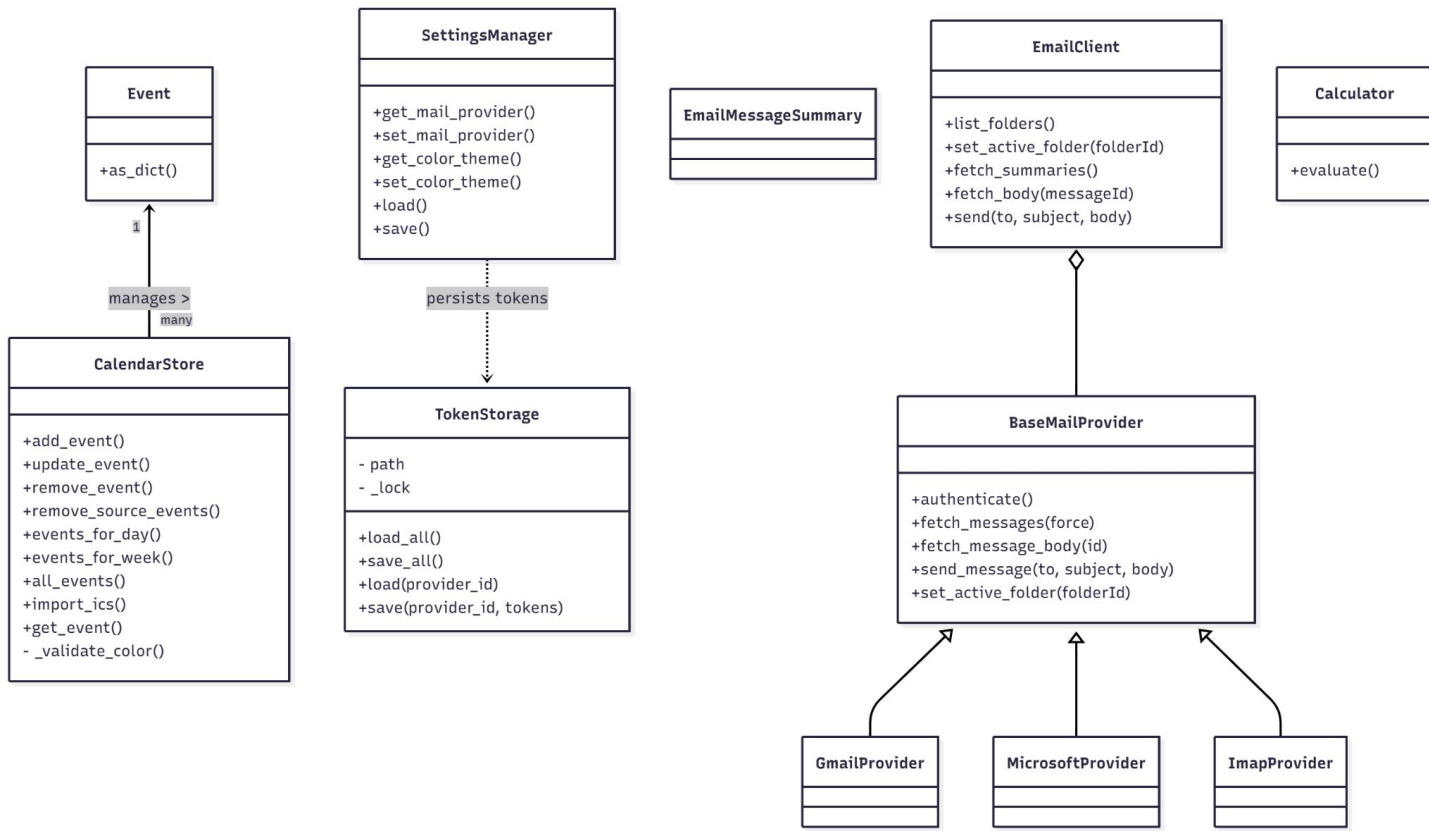
Diagram klas UML aplikacji StudyHub, widok logiczny

Hierarchia i zależności pomiędzy głównymi modułami aplikacji:

- Warstwa UI (interfejs użytkownika) – klasy MainWindow, CalendarView, MailView, NotesView, SettingsView, Sidebar,
- warstwa logiki (core) – CalendarStore, SettingsManager, TokenStorage,
- Warstwa obsługi poczty i synchronizacji – EmailClient, BaseMailProvider, GmailProvider, MicrosoftProvider, ImapProvider.



Core (modele i serwisy)



Core (modele i serwisy)

Diagram przedstawia strukturę klas warstwy logiki (core).

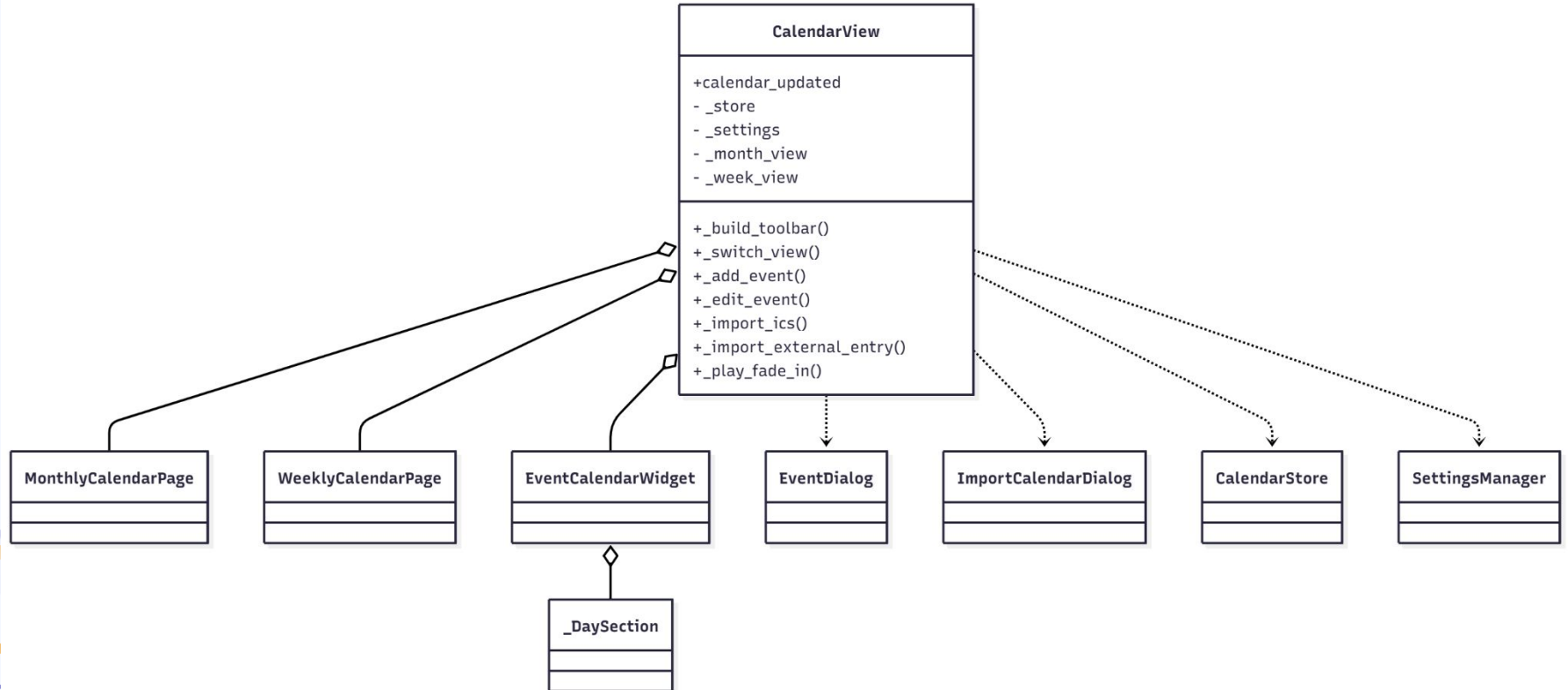
Ukazuje zależności pomiędzy modułami odpowiedzialnymi za zarządzanie danymi, ustawieniami oraz obsługę poczty elektronicznej.

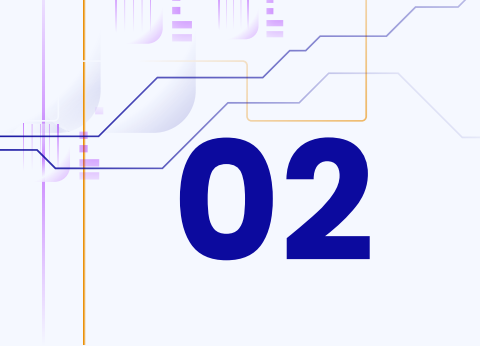
Moduł kalendarza (CalendarStore) zarządza obiektami klasy Event – dodawaniem, usuwaniem i importem wydarzeń.

Moduł ustawień (SettingsManager) przechowuje konfigurację aplikacji i współpracuje z klasą TokenStorage, odpowiedzialną za zapisywanie danych logowania do kont pocztowych.

Klasy związane z obsługą poczty – EmailClient, który korzysta z abstrakcyjnej klasy bazowej BaseMailProvider oraz jej implementacji: GmailProvider, MicrosoftProvider i ImapProvider.

Kalendarz





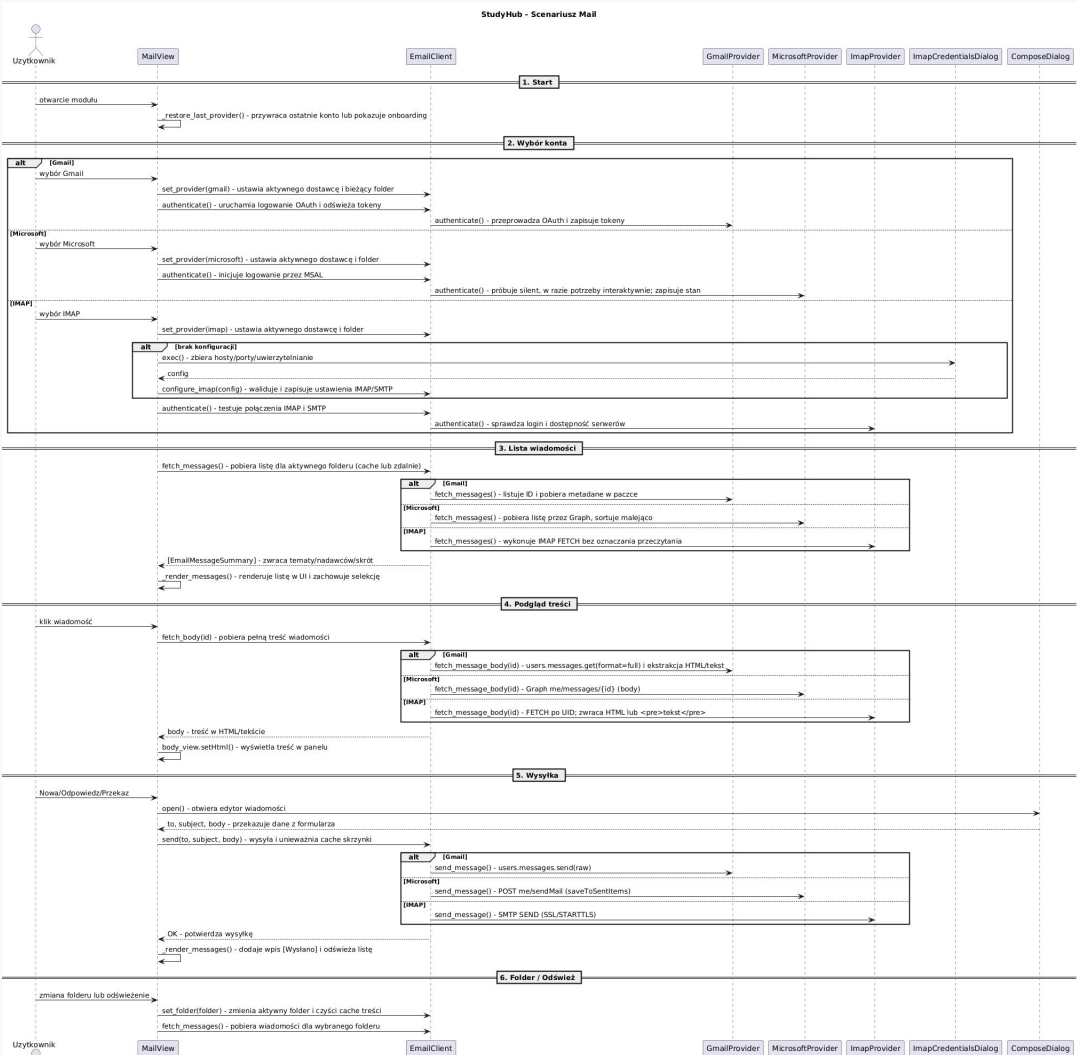
02

Widok procesu

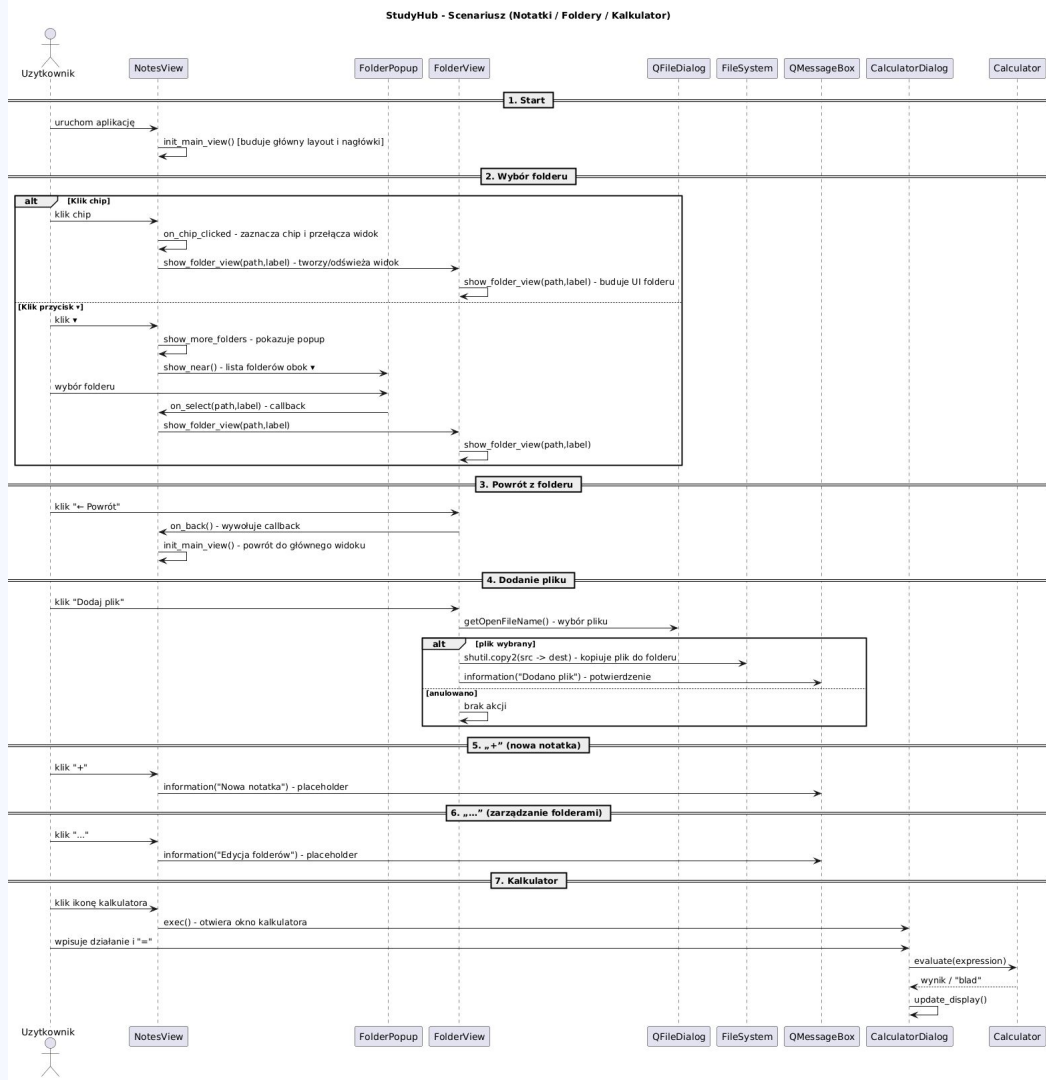
(UML Communication / Sequence Diagram)



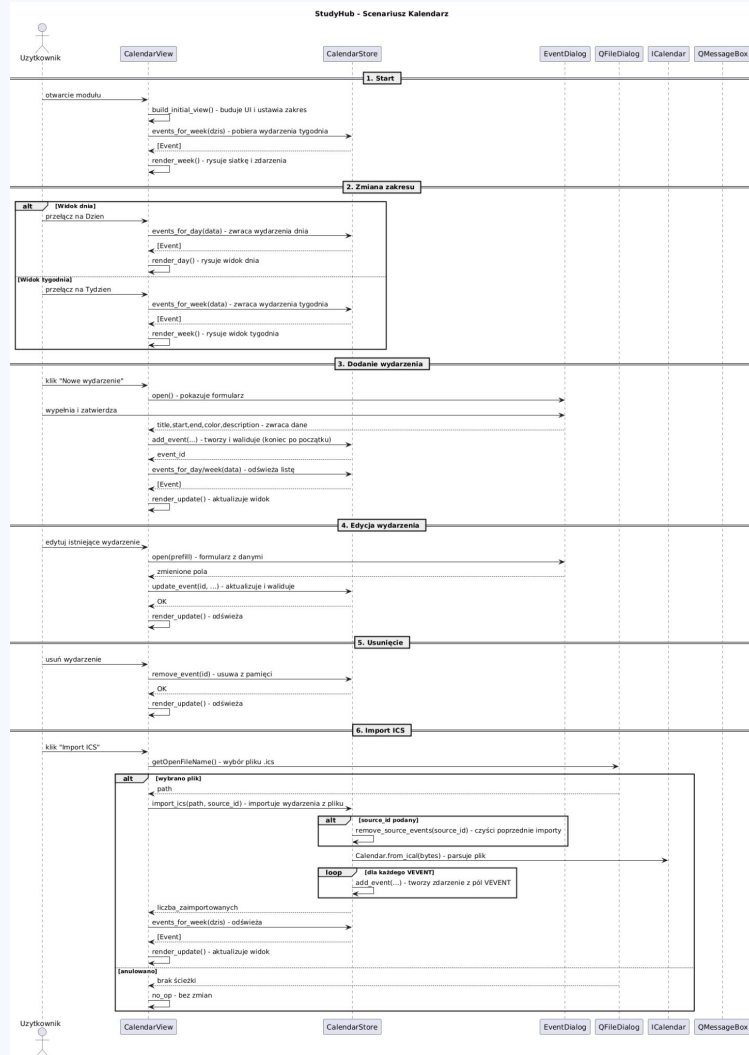
UML Sequence diagram- scenariusz mail



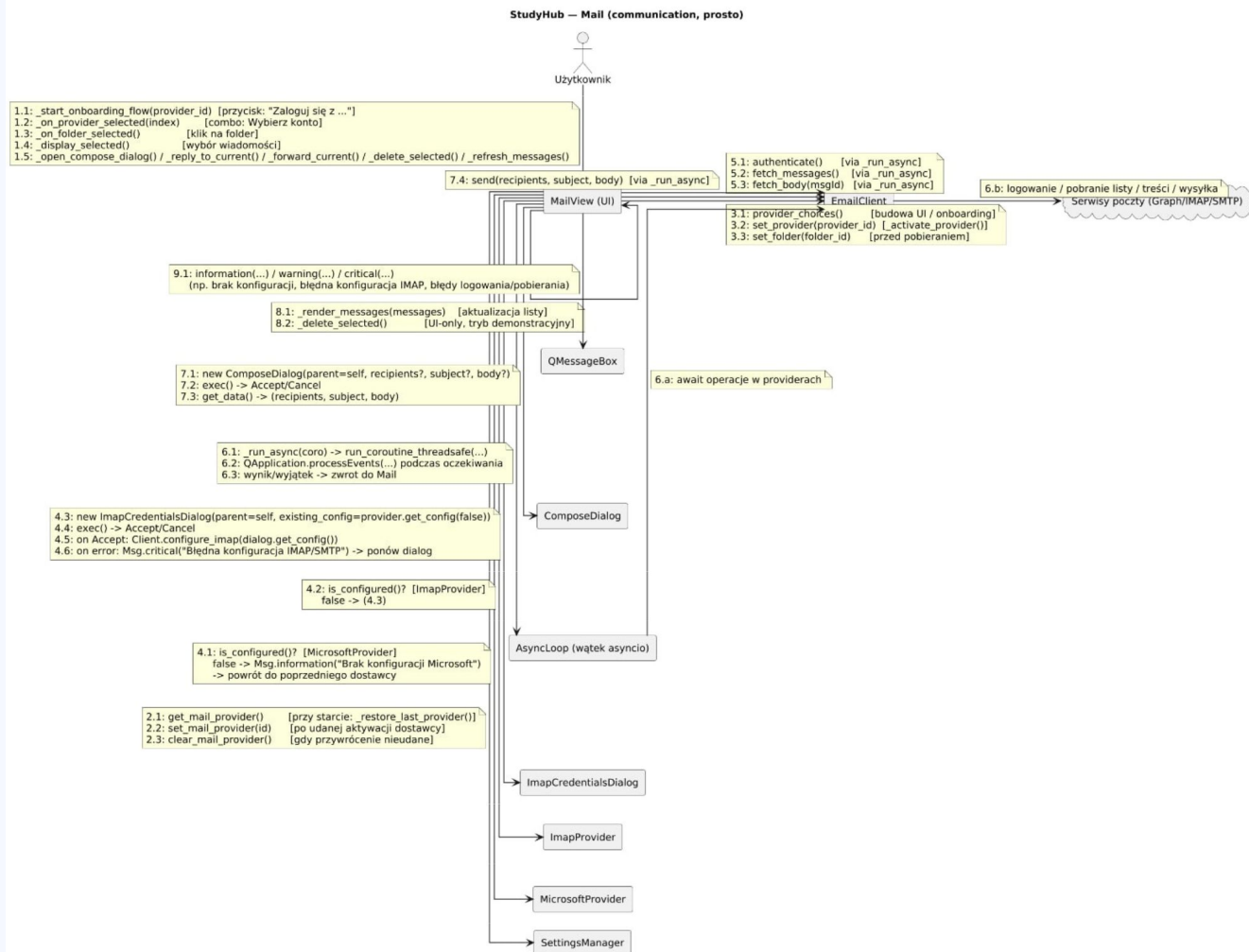
UML Sequence diagram- scenariusz notatki



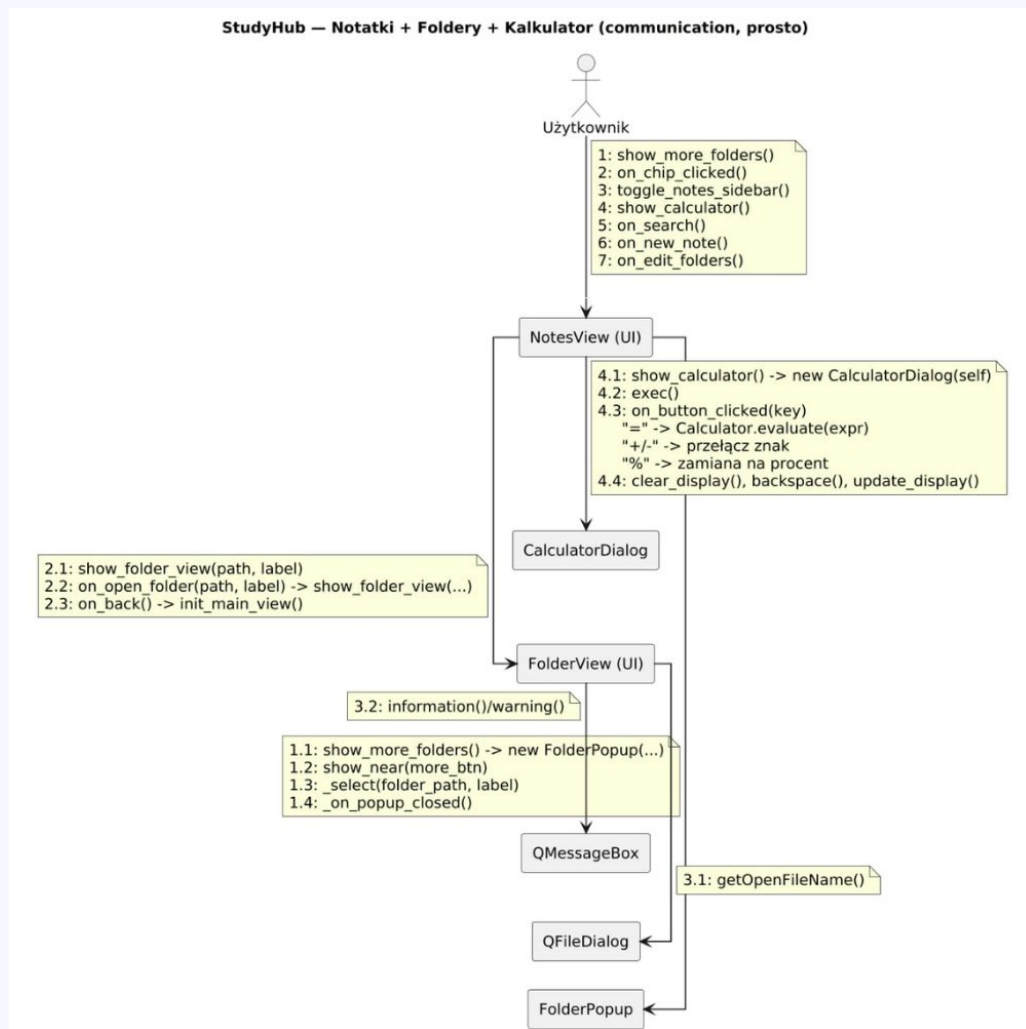
UML Sequence diagram- scenariusz kalendarz



UML Communication Diagram-scenariusz mail

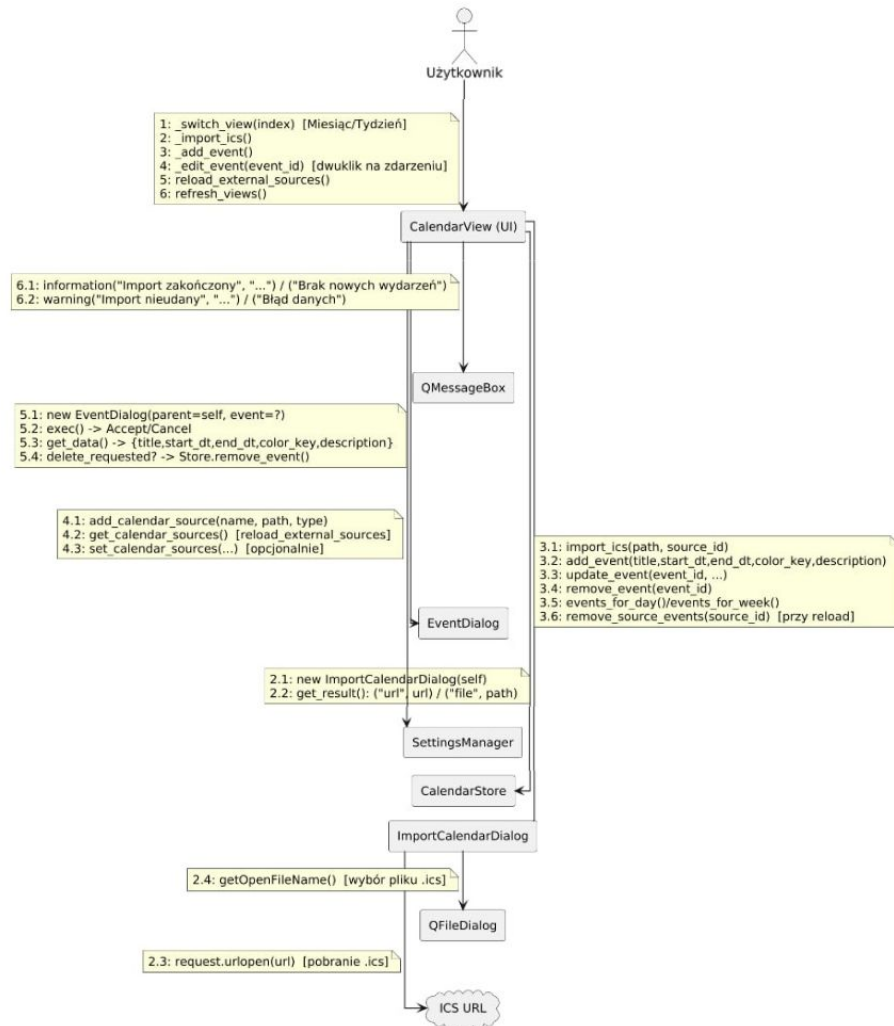


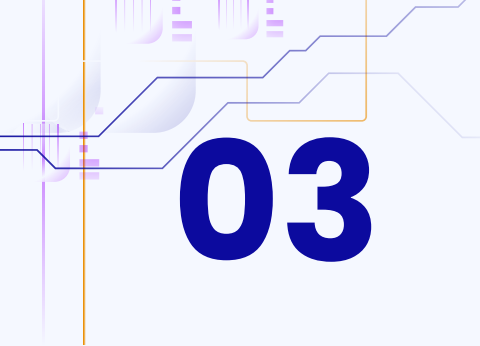
UML Communication Diagram-scenariusz notatki, kalkulator



UML Communication Diagram-scenariusz kalendarz

StudyHub — Kalendarz (communication, prosto)





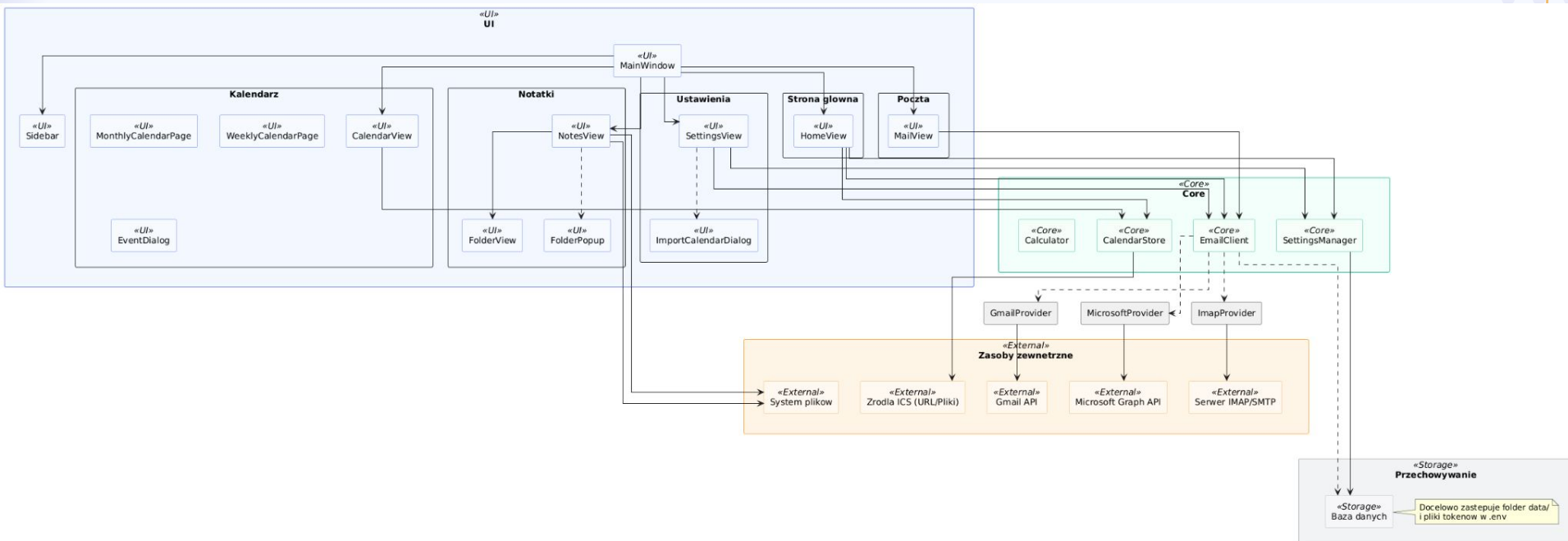
03

Widok developerski

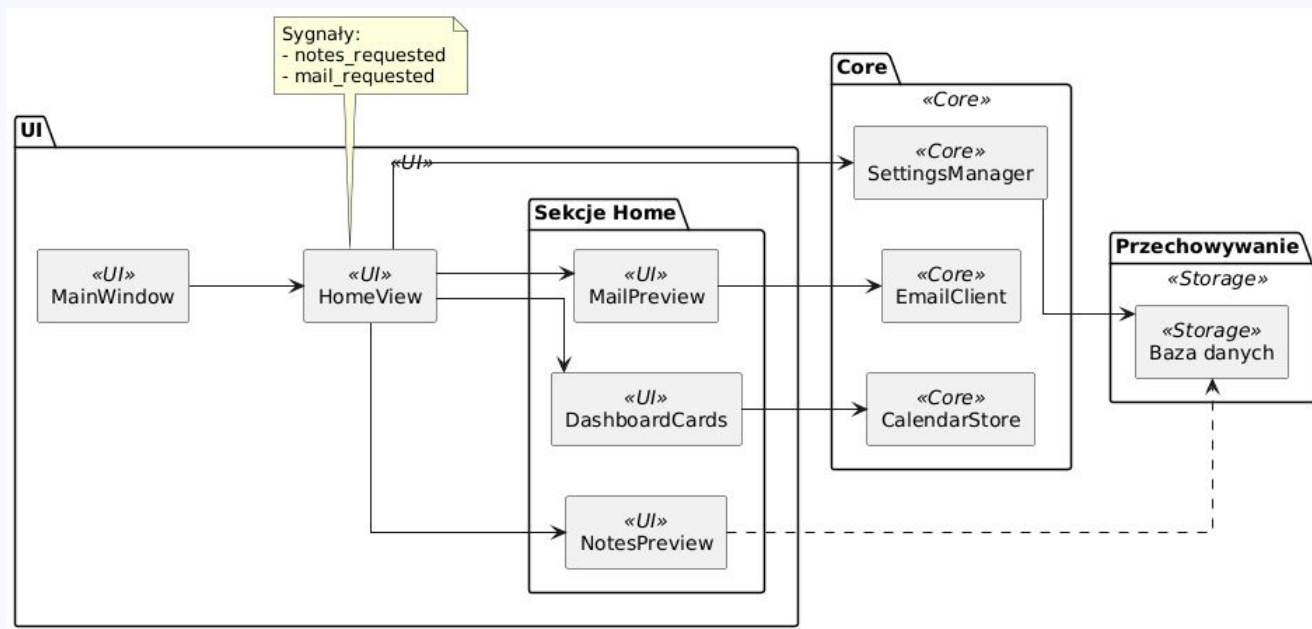
(UML Component diagram)



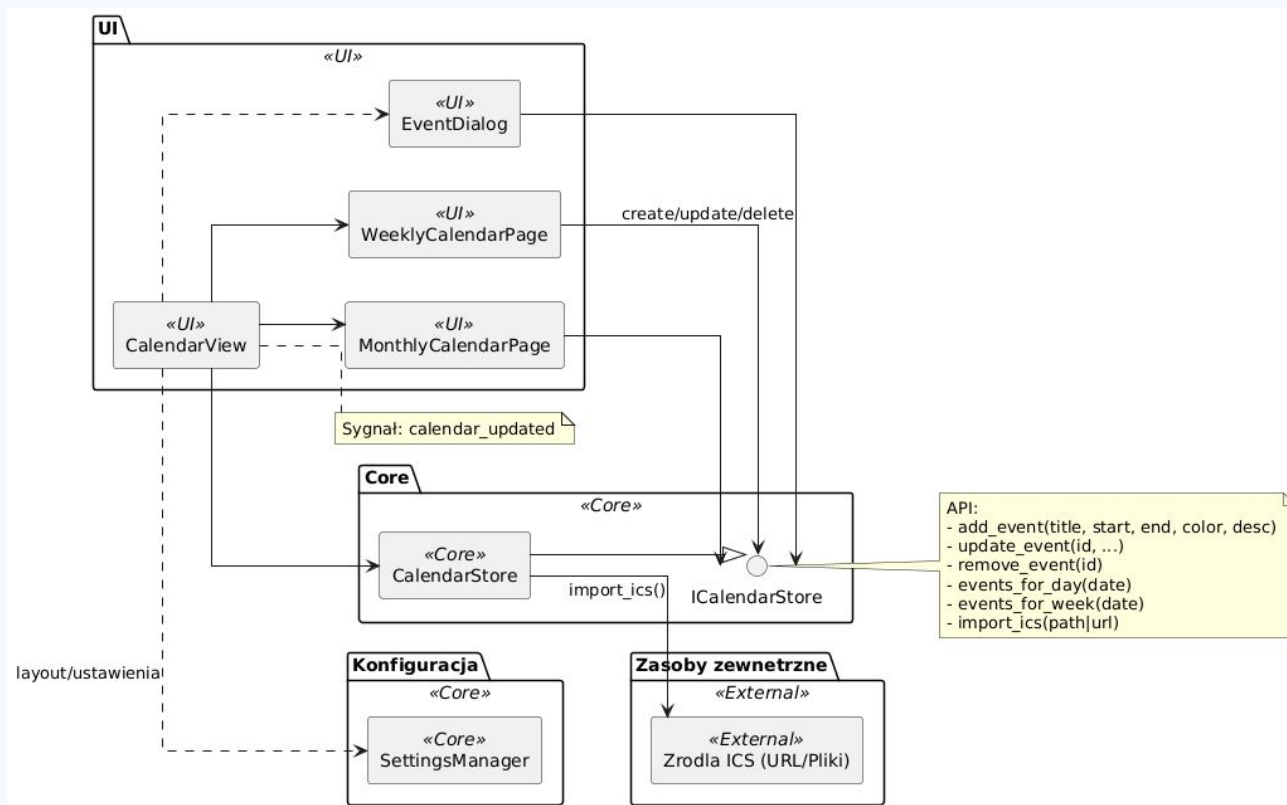
Widok developerski - ogólny



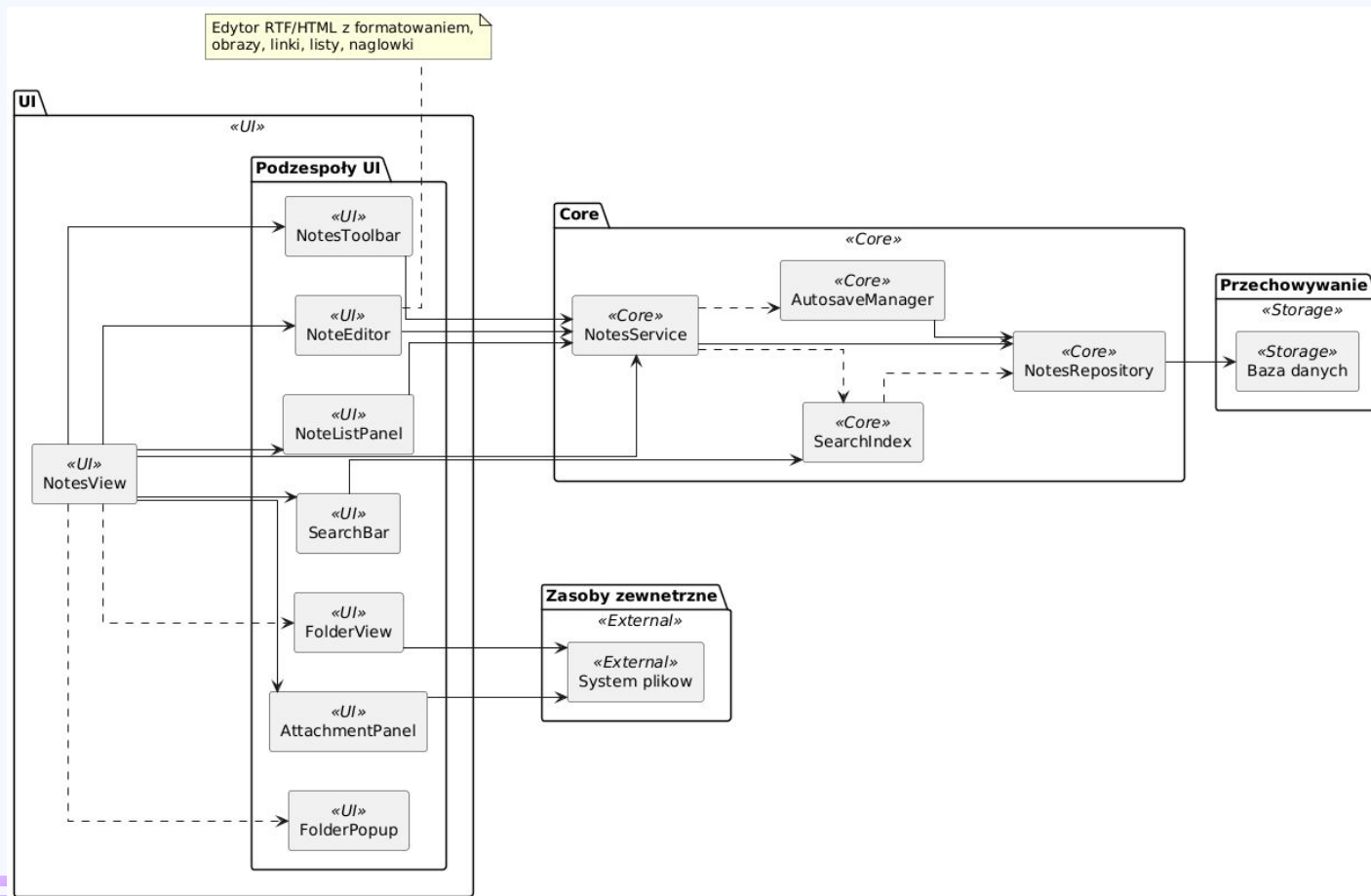
Widok developerski – strona główna



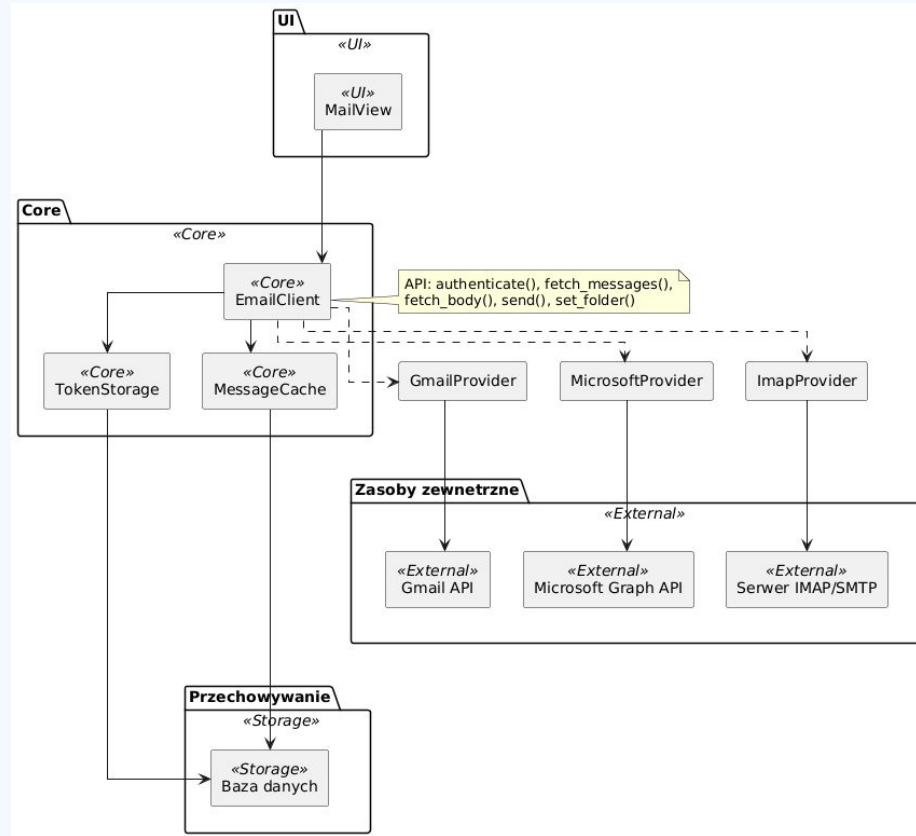
Widok developerski - kalendarz



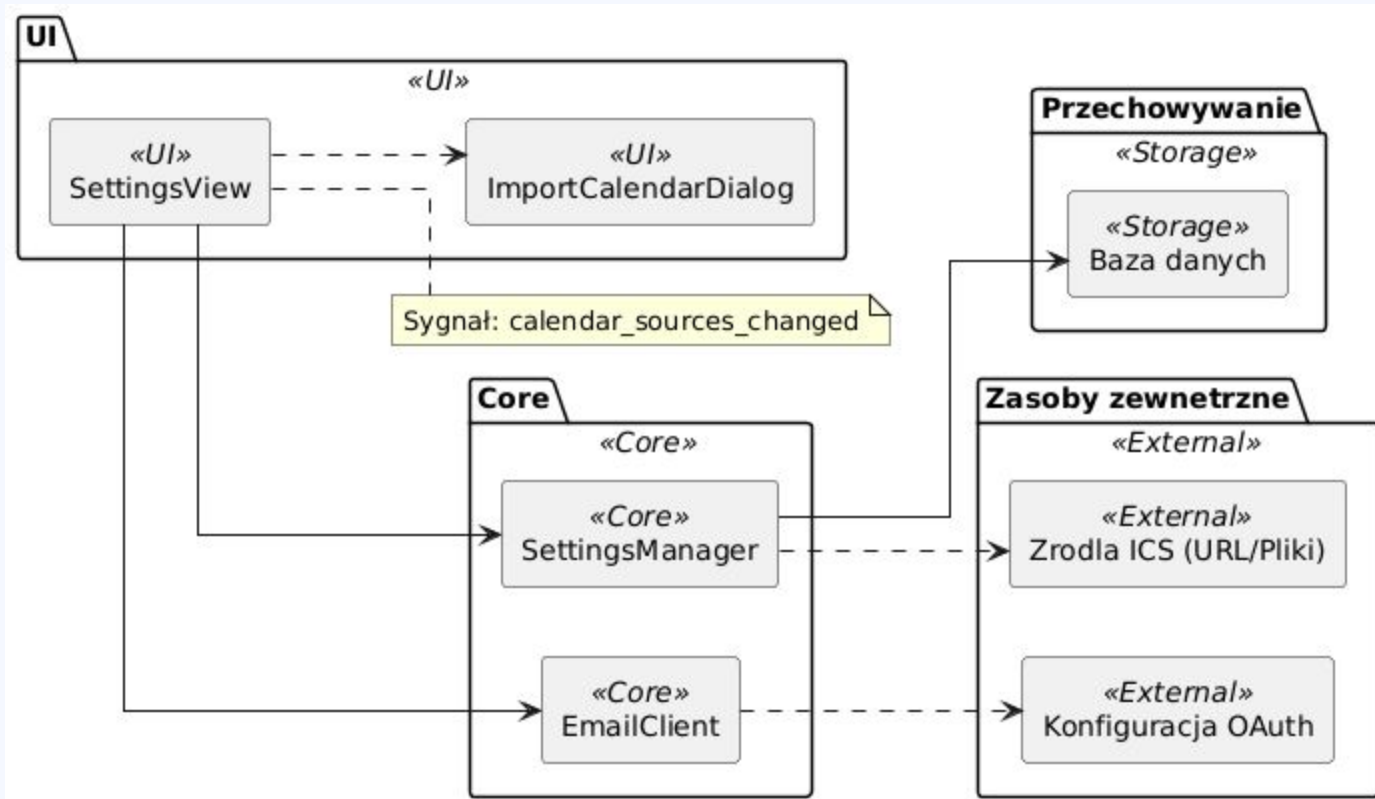
Widok developerski - notatki



Widok developerski - mail



Widok developerski - ustawienia



04

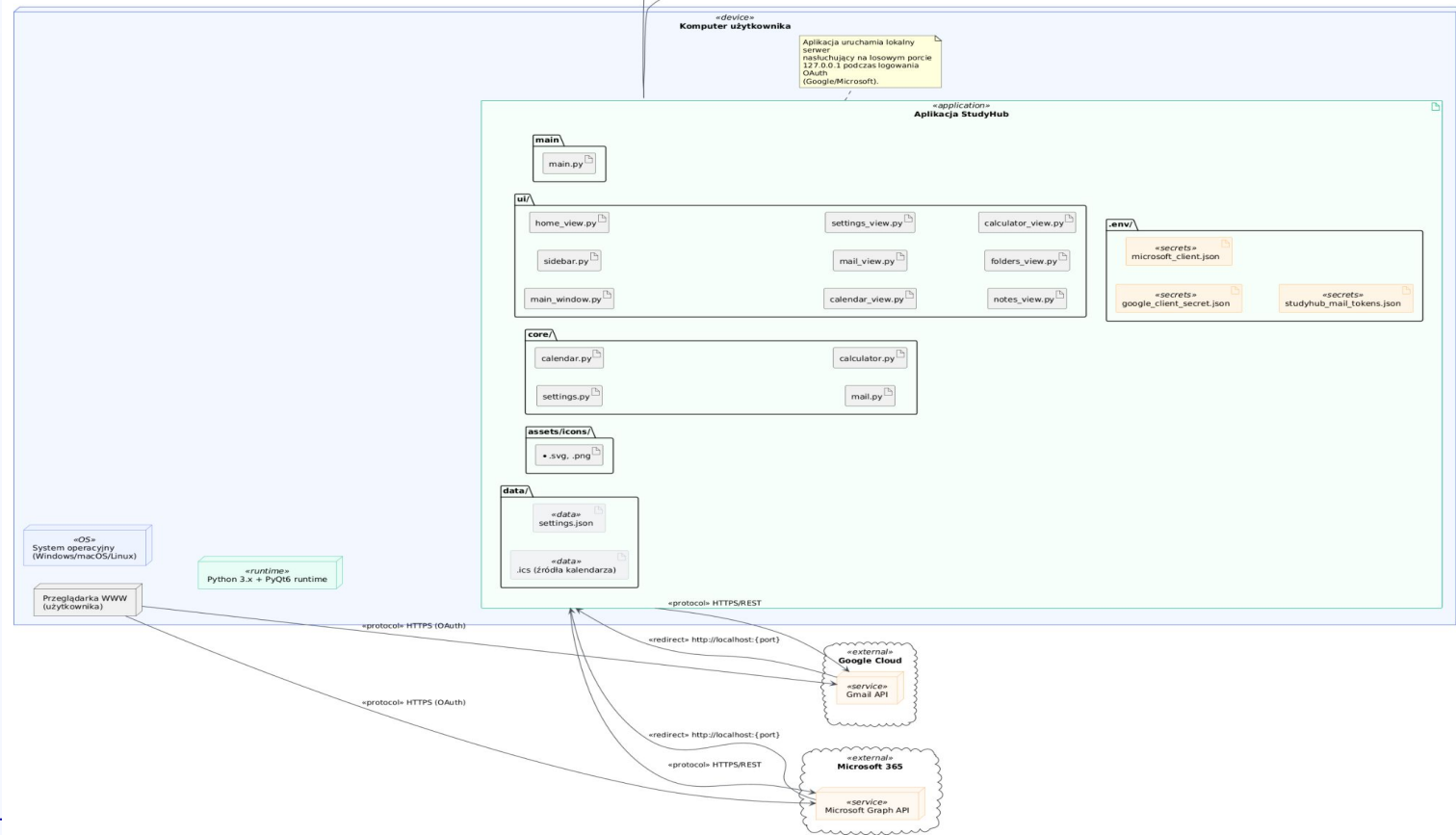
Widok fizyczny (UML Deployment diagram)



StudyHub — Widok fizyczny (Deployment)

Opis połączeń

- «protocol» HTTP/HTTPS — połączenia sieciowe
- «protocol» IMAPS/SMTPS — szyfrowane połączenia pocztowe
- «redirect» localhost — zwrot z logowania OAuth do aplikacji





05

Architecture design records



ADR cz. I

1. Kontekst:

StudyHub to desktopowa aplikacja edukacyjna (Python 3.12 + PyQt6) obejmująca moduły: Mail, Calendar (ICS), Notes, Settings, System rozplanowywania nauki (w przyszłości). Modułarny GUI oparty o nawigację boczną (sidebar + QStackedWidget). Kod dzieli się na warstwy: ``core/`` (logika) i ``ui/`` (widoki PyQt6).

2. Framework GUI: PyQt6

Decyzja: PyQt6.

Uzasadnienie: natywne komponenty (QMainWindow, QFrame, QStackedWidget), szybka integracja z logiką, brak potrzeby backendu.

Konsekwencje: tylko lokalne środowisko; wymagane zależności Pythona; szybkie prototypowanie.

Alternatywa odrzucona: React.

3. Struktura modułowa (MVC-inspired)

Decyzja: oddzielenie ``core/`` (modele, integracje: mail, calendar, settings) od ``ui/`` (widoki: MailView, CalendarView, NotesView).

Uzasadnienie: czytelność, testowalność, możliwość wymiany warstwy prezentacji.

Konsekwencje: konieczność stabilnych interfejsów (np. EmailClient API, CalendarStore API).

Powiązanie z kodem: ``ui/mail_view.py`` korzysta z ``core.mail.EmailClient``; ``ui/notes_view.py`` izoluje logikę UI (brak zapisu jeszcze – do rozszerzenia w przyszłych ADR).

ADR cz. II

4. Konfiguracja i stan użytkownika

Decyzja: JSON (`data/settings.json`) dla preferencji (wybrany provider poczty, źródła kalendarza, motyw).

Uzasadnienie: prosty, przenośny format.

Ryzyko: brak szyfrowania tokenów/OAuth.

Alternatywy: SQLite (wprowadzamy w przyszłości)

5. Poczta (OAuth + IMAP/SMTP)

Decyzja: Gmail/Microsoft przez OAuth + IMAP/SMTP (async).

Uzasadnienie: zgodność z politykami bezpieczeństwa; brak przechowywania haseł.

Implementacja: w `MailView` oddzielny wątek z pętlą asyncio (`\_run\_async`) do asynchronicznych operacji (fetch, send).

Konsekwencje: konieczność konfiguracji plików klienta (Microsoft/Gmail), obsługa błędów w UI (QMessageBox).

Do doprecyzowania: mechanizm odświeżania tokenów + bezpieczne składowanie.

ADR cz. III

6. Kalendarz – ICS

Decyzja: format ICS + biblioteka icalendar + import z linku.

Uzasadnienie: interoperacyjność (Google/Outlook/Apple) + zależności online.

Konsekwencje: aktualizacja przez import.

7. Nawigacja (Sidebar + QStackedWidget)

Decyzja: centralny MainWindow z QStackedWidget + boczny panel wyboru modułu.

Uzasadnienie: spójny UX, łatwa rozbudowa (dodanie nowego widoku = nowy QWidget).

Konsekwencje: prosty routing wewnętrzny; brak wielookienkowego chaosu.

8. Styl/UI

Decyzja: jasny, neutralny motyw (biel/szarość + akcent niebieski).

Uzasadnienie: czytelność treści, brak dystrakcji.

Konsekwencje: łatwe późniejsze wprowadzenie motywów



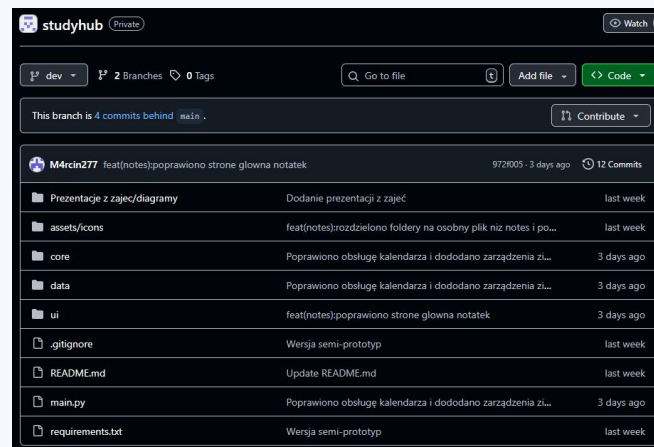
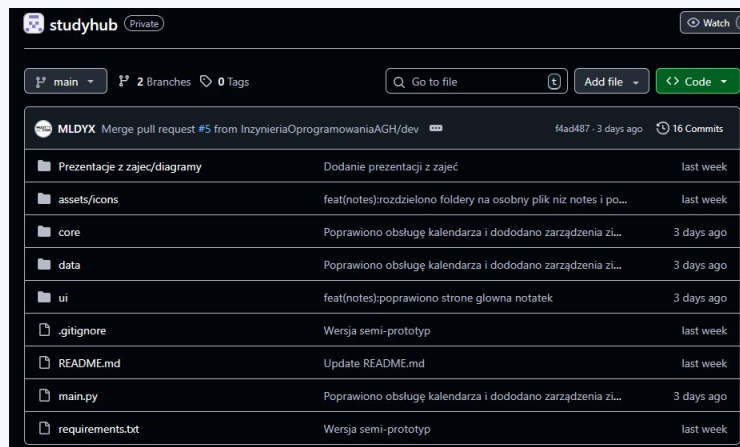
06

Repozytorium Github



Link do repozytorium git:

<https://github.com/InzynieriaOprogramowaniaAGH/studyhub>



Lint i formater:

W tym celu używamy ruff'a dla którego "wytyczne" zdefiniowane są w pliku pyproject.toml

```
pyproject.toml
1 [tool.ruff]
2 target-version = "py311"
3 line-length = 100
4 src = ["."]
5 extend-exclude = ["Prezentacje z zajec", "__pycache__"]
6
7 [tool.ruff.lint]
8 select = ["F", "E", "W", "B", "I"]
9
10 [tool.ruff.format]
11 quote-style = "double"
12 indent-style = "space"
```



07

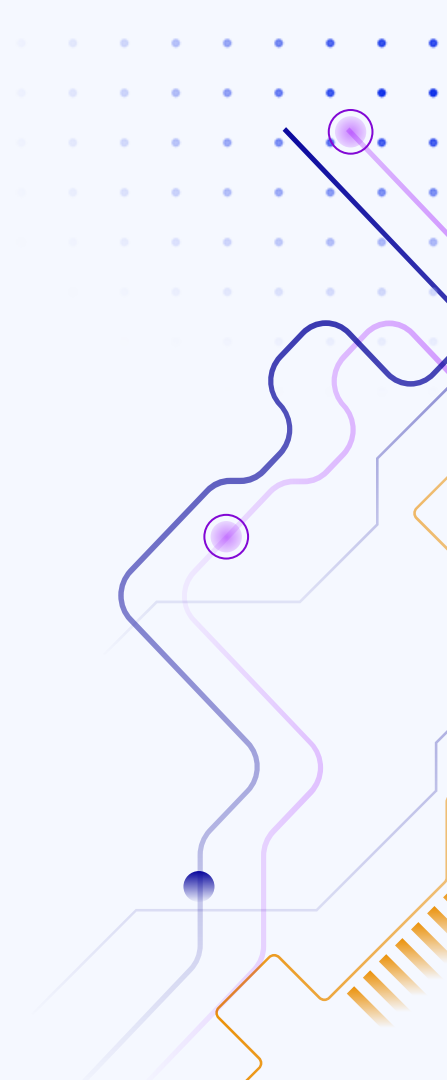
Implementacja

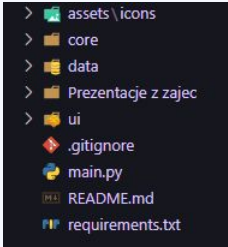


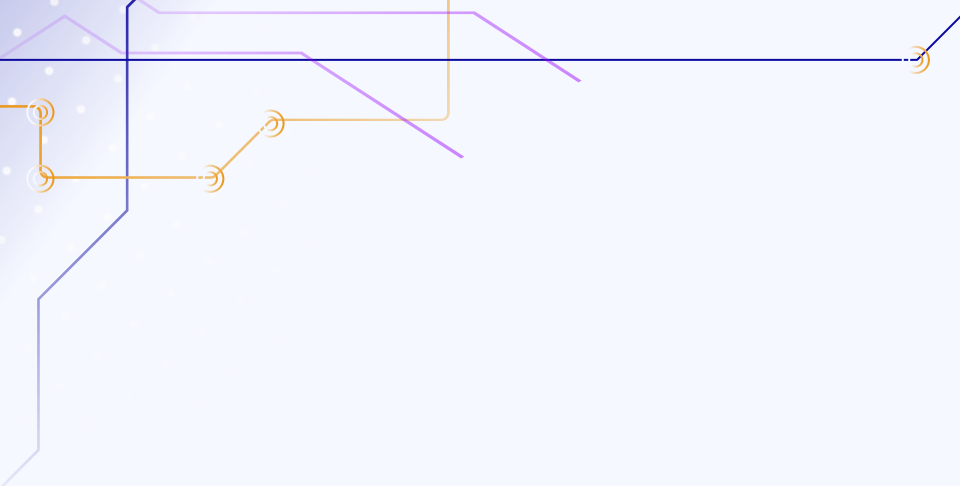


08

AI - Prompt



Po co AI	Jaki Prompt	Wynik
Claude Sonnet 4 wykorzystany to zaproponowania najlepszej struktury projektu dla łatwości pracy i czytelności.	"Zaproponuj kompletną i dobrze zorganizowaną strukturę projektu w Pythonie z wykorzystaniem PyQt6. Projekt ma być czytelny, łatwy do rozbudowy oraz utrzymania w 3-osobowym zespole. Uwzględnij logiczny podział katalogów i plików, sposób zarządzania interfejsem użytkownika, oddzielenie logiki aplikacji od warstwy prezentacji. Możesz zasugerować wzorce projektowe, konwencje nazewnictwa i metody inicjalizacji aplikacji, tak aby całość była łatwa w obsłudze."	Obecna struktura projektu :)  <pre> > assets\icons > core > data > Prezentacje z zajęć > ui > .gitignore > main.py > README.md > requirements.txt </pre>
ChatGPT-5 wykorzystany do polecenia kolorystyki aplikacji tak aby była przyjemna do użytkowania.	"Zaproponuj jasne kolorystyki dla aplikacji tworzonej w Pythonie z użyciem PyQt 6. Styl ma być nowoczesny, przyjemny w użytkowaniu i estetyczny przy dłuższej pracy z interfejsem. Uwzględnij kolory tła, akcentów, przycisków, tekstu i ikon."	Uzyskaliśmy 3 propozycje kolorowe. 2 pierwsze totalnie nie zdatne do użytkowania (po wypróbowaniu oczy bolały na samą myśl o tym interfejsie.) Ostatni po lekkich modyfikacjach używamy obecnie w projekcie
ChatGPT-5 wykorzystywany do pisania tekstów "placeholderów" w aplikacji.	Tutaj było parę promptów ale przykładowy to: "Utwórz krótkie 4 notatki (tytuł + 1 zdanie), które posłużą jako placeholder podglądu skrótowego w naszej aplikacji"	Teksty obecnie znajdujące się w projekcie. Są to "placeholderzy" na czas implementacji UI.
Claude Sonnet 4.5 wykorzystany w celu zaproponowania metod implementacji maila.	"Podaj najlepsze metody do implementacji własnego "clienta" mailowego zawierającego takie foldery i opcje jak odebrane, wysłane, wyslij wiadomość, przekaż dalej itp. Jak pełnoprawny mail. Chodzi o metodą w której nie trzeba tego pisać od 0 tylko istnieje możliwość wykorzystania istniejących już technologii"	Obecny plan budowy maila. Możliwość zalogowania przez Google / M365 / IMAP / SMTP. (Google i Microsoft365 zewnętrznie do gmail i outlook). IMAP / SMTP przykładowo do roundcube (poczta agh).

A decorative graphic in the top-left corner featuring a network of thin, intersecting lines in blue, purple, and orange. Some lines terminate in small circular nodes, resembling a circuit board or a data network diagram.

KONIEC

Wiktor Rzepka, Aleksandra Suder, Marcin Stępkowski

A decorative graphic in the bottom-right corner featuring a grid of small blue dots. Overlaid on this grid are several wavy, flowing lines in orange and blue, along with some geometric shapes like triangles and squares, creating a modern, abstract design.